Modul mahasiswa Lab 07 — BaaS, Auth, RLS, dan pgvector

Kebijakan kelas: Lab ini latihan formatif, tanpa tugas, nilai, atau penyerahan terpisah. Satu proyek besar dikerjakan oleh kelompok **3 orang**, dengan presentasi checkpoint minggu 7 (UTS) dan hasil akhir minggu 14 (UAS). Simpan hasil lab hanya bila berguna sebagai referensi atau bukti proses proyek. Baca brief proyek kelompok. Bobot resmi tetap mengikuti RPS/LMS.

COMP6991031, sesi 07. Modul ini memuat langkah dari awal sampai akhir, **kunci seluruh challenge A–F**, dan screenshot yang menerangkan perintah, fungsi, cara kerja, serta hasilnya. Jalur wajib berjalan lokal dengan Docker Compose. Jalur Supabase membutuhkan akun/proyek dan bersifat tambahan. Jalankan perintah dari **root repo meet7CloudService**.

Kasus kerja dan hasil belajar

Dua staf, Alice dan Bob, menyimpan catatan dalam satu database. Jika frontend keliru meminta data staf lain, database tetap harus menyaringnya. Tim juga ingin mencari dokumen yang mirip untuk fitur AI berikutnya. Setelah lab kamu dapat menjelaskan perbedaan Auth dan otorisasi, `GRANT` dan policy RLS, `USING` dan `WITH CHECK`, identitas `current_user` versus `auth.uid()`, serta tipe `vector` dan operator jarak `<->`.

Komponen	Fungsi pada kasus
PostgreSQL `db`	Menyimpan catatan dua staf di satu tabel.
`lab_alice` / `lab_bob`	Identitas lokal untuk membuktikan RLS tanpa akun cloud.
Policy `rls_demo_owner`	Menyaring baris yang boleh dibaca/ditulis setiap identitas.
Adminer	UI inspeksi tabel; login `labadmin` adalah **pemilik**, jadi dapat melihat kedua baris.
pgvector	Menyimpan dan membandingkan vektor; contoh Lab 07 memakai tiga dimensi mainan.
Supabase	Jalur cloud tambahan: Auth JWT, `auth.uid()`, SQL Editor, dan Storage.

Batas bukti: Screenshot terminal gelap adalah cuplikan output uji lokal aktual yang ditata agar mudah dibaca. Screenshot Adminer berasal dari Chrome saat container lab hidup. Nama container, waktu, ID, dan versi patch di laptopmu dapat berbeda. Simpan screenshot **hasil praktik sendiri** bila bermanfaat untuk proyek kelompok.

0. Siapkan repo dan database

Pastikan Docker Desktop menunjukkan **Engine running**. Jika memakai repo kelas, buka SeedFlora/meet7CloudService, pilih **Use this template**, lalu clone repo salinanmu. Dari root repo, salin contoh password menjadi file lokal. Nilainya hanya untuk database latihan; ubah nilainya bila perlu dan jangan commit file ini.

PowerShell:

```
Copy-Item .\secrets\db_password.example.txt .\secrets\db_password.txt
docker version
docker compose config --quiet
docker compose --profile debug up -d --wait
docker compose ps
```

Bash/Linux/Git Bash:

```
cp secrets/db_password.example.txt secrets/db_password.txt
docker version
docker compose config --quiet
docker compose --profile debug up -d --wait
docker compose ps
```

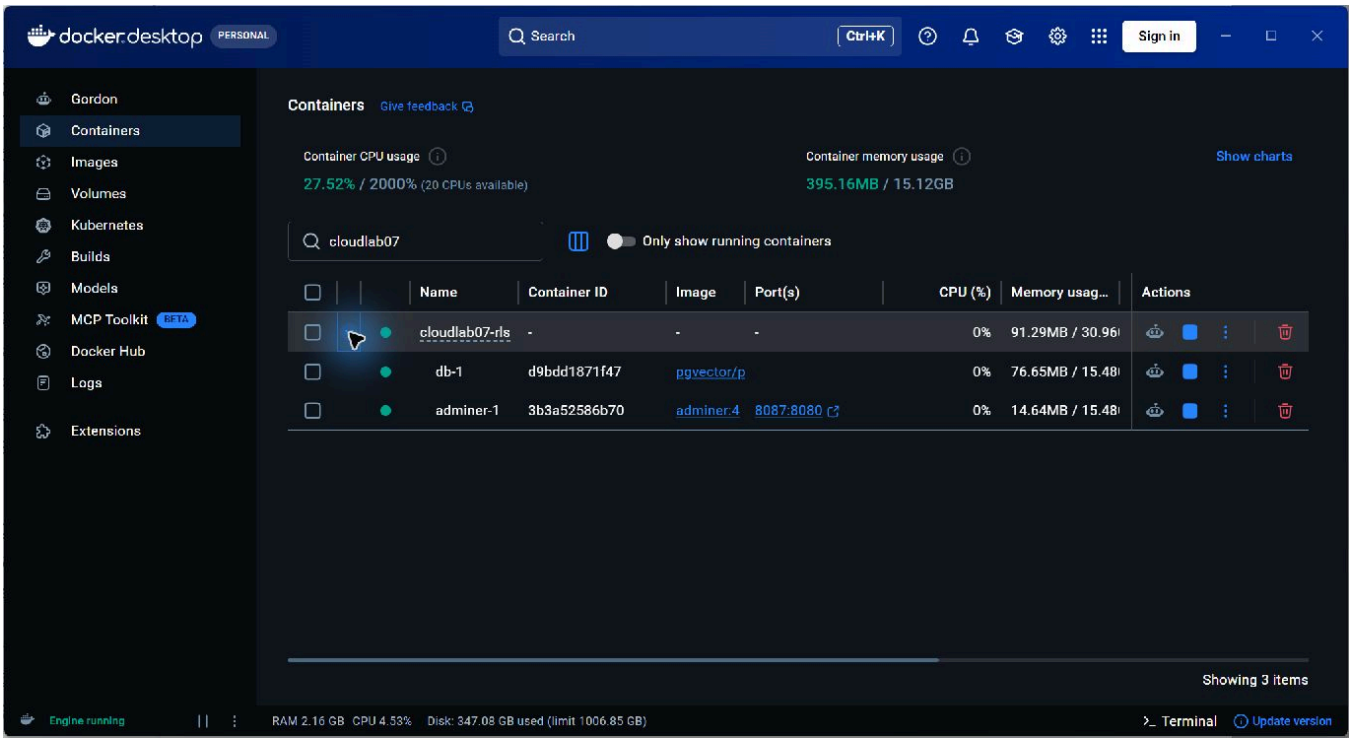
`config --quiet` memeriksa sintaks Compose. `up -d --wait` membuat database dan Adminer, lalu menunggu healthcheck database. Profile `debug` menerbitkan Adminer hanya di `127.0.0.1:8087`; port PostgreSQL tidak diterbitkan ke host. `ps` harus menunjukkan `db` sehat dan `adminer` berjalan.



Langkah: jalankan `docker version`, `docker compose config --quiet`, lalu `git check-ignore -v secrets/db_password.txt`. Fungsi: memastikan Docker tersedia, YAML valid, dan password lokal tidak masuk commit. Cara kerja: Docker melaporkan versi server; Compose memvalidasi model tanpa menjalankan container; Git mencocokkan path terhadap `.gitignore`. Baca hasil: semua command exit 0, `Compose config: valid`, dan aturan ignore untuk file password tampil. Ini render dari output run aktual, bukan screenshot terminal mentah.



Langkah: jalankan `docker compose --profile debug up -d --wait` dan `docker compose ps`. Fungsi: memastikan database siap sebelum SQL dijalankan. Cara kerja: Compose membuat jaringan serta volume lab, menunggu `pg_isready`, dan memetakan port Adminer 8087 ke 8080 dalam container. Baca hasil: `db` Healthy dan `adminer` Up; port 5432 hanya internal.



Langkah: buka tab Containers di Docker Desktop dan perluas proyek `cloudlab07-rls` *setelah* `docker compose --profile debug up -d --wait`. *Fungsi: memeriksa layanan melalui tampilan visual. Cara kerja: Compose mengelompokkan container* `db-1` *dan* `adminer-1` *dalam satu proyek; titik hijau berarti container berjalan. Baca hasil: image* `pgvector/pgvector:pg16` *untuk database dan* `adminer:4` *pada port host 8087. Kesehatan database tetap dikonfirmasi oleh* `docker compose ps`.

1. Jalankan SQL dan baca policy RLS

`local-rls-demo.sql` membuat dua role, tabel `rls_demo`, hak akses, policy, dan dua catatan contoh. Script mengosongkan **hanya** tabel demo itu saat dijalankan ulang. Jangan menaruh data penting di tabel latihan.

PowerShell:

```
Get-Content -Raw .\local-rls-demo.sql | docker compose exec -T db psql -v ON_ERROR_STOP=1 -U labadmin
-d lab07
```

Bash/Linux/Git Bash:

```
docker compose exec -T db psql -v ON_ERROR_STOP=1 -U labadmin -d lab07 < local-rls-demo.sql
```

`-T` mematikan pseudo-TTY agar input pipe/redirection diterima. `ON_ERROR_STOP=1` membuat command gagal bila SQL salah. `labadmin` dipakai untuk setup karena ia pemilik tabel; uji pembatasan nanti memakai `SET ROLE`.

LAB 07 | 10 SQL SETUP

```
$ Get-Content -Raw .\local-rls-demo.sql | docker compose exec -T db psql -v
ON_ERROR_STOP=1 -U labadmin -d lab07
DO
CREATE TABLE
ALTER TABLE
GRANT
DROP POLICY
CREATE POLICY
TRUNCATE TABLE
SET
INSERT 0 1
  viewer   | owner_name |     note
-----------+------------+--------------
 lab_alice | lab_alice  | Catatan Alice
(1 row)

RESET
SET
INSERT 0 1
 viewer  | owner_name |    note
---------+------------+-------------
 lab_bob | lab_bob    | Catatan Bob
(1 row)

RESET
NOTICE:  relation "rls_demo" already exists, skipping
[exit 0]
```

Output perintah aktual ditata ulang agar terbaca; bukan tangkapan layar terminal mentah.

Langkah: pipe `local-rls-demo.sql` ke `psql` dalam container `db`. Fungsi: membuat role, tabel, GRANT, policy RLS, dan data uji. Cara kerja: `psql` mengeksekusi statement berurutan dan berhenti saat error karena `ON_ERROR_STOP=1`. Baca hasil: `CREATE POLICY`, dua `INSERT 0 1`, dan satu baris Alice/Bob pada query uji; `NOTICE` bila tabel sudah ada adalah aman saat mengulang. Output lengkap.

Periksa policy yang benar-benar tersimpan:

```
docker compose exec -T db psql -U labadmin -d lab07 -c "SELECT policyname, cmd, roles, qual,
with_check FROM pg_policies WHERE tablename='rls_demo';"
```

LAB 07 ? policy RLS

```
$ docker compose exec -T db psql -U labadmin -d lab07 -c "SELECT ... FROM pg_policies"
policyname    | cmd |       roles        |           qual            |         with_check
--------------+-----+--------------------+---------------------------+----------------------------
 rls_demo_owner | ALL | {lab_alice,lab_bob} | (owner_name = CURRENT_USER) | (owner_name = CURRENT_USER)
(1 row)
```

Cuplikan output uji lokal aktual; baris dipilih dan ditata agar terbaca.

Langkah: jalankan query `pg_policies`. Fungsi: melihat aturan yang digunakan PostgreSQL. Cara kerja: `USING` menentukan baris yang terlihat/dapat diubah; `WITH CHECK` memeriksa pemilik baris baru/hasil perubahan. Baca hasil: kedua ekspresi membandingkan `owner_name` dengan `CURRENT_USER` untuk role Alice dan Bob.

Sekarang baca tabel dari dua identitas yang berbeda. Ini command yang sama pada PowerShell dan Bash:

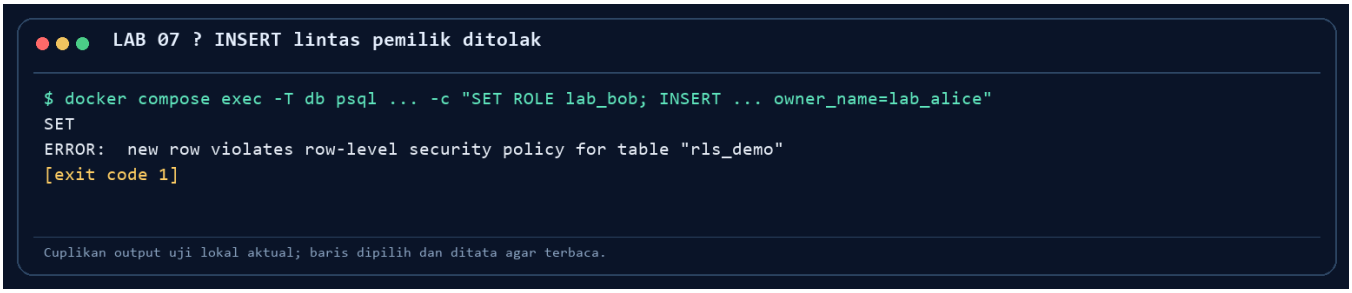
```
docker compose exec -T db psql -v ON_ERROR_STOP=1 -U labadmin -d lab07 -c "SET ROLE lab_alice; SELECT
current_user AS viewer, owner_name, note FROM rls_demo; RESET ROLE; SET ROLE lab_bob; SELECT
current_user AS viewer, owner_name, note FROM rls_demo;"
```



Langkah: jalankan `SET ROLE lab_alice` *lalu* `lab_bob` *sebelum* `SELECT`*. Fungsi: membuktikan pemisahan data dalam satu tabel. Cara kerja: policy RLS menambahkan filter menurut* `current_user` *pada query tiap role. Baca hasil: Alice melihat Catatan Alice, Bob melihat Catatan Bob; tidak ada baris silang.*

Uji penolakan tulis. Command ini **sengaja berakhir dengan exit code bukan nol**:

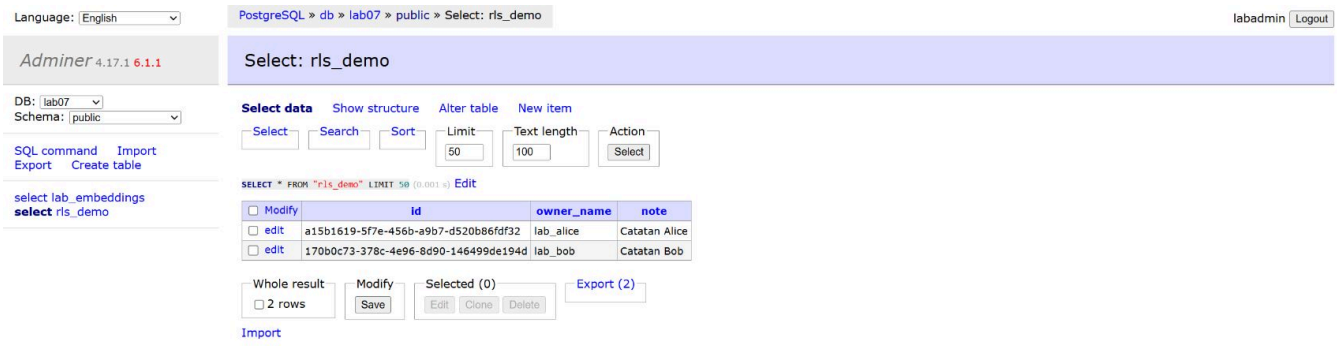
```
docker compose exec -T db psql -v ON_ERROR_STOP=1 -U labadmin -d lab07 -c "SET ROLE lab_bob; INSERT INTO rls_demo (owner_name,note) VALUES ('lab_alice','Percobaan silang');"
```



Langkah: sebagai Bob, coba membuat baris berlabel Alice. Fungsi: menguji batas otorisasi saat aplikasi salah mengirim pemilik. Cara kerja: policy `WITH CHECK` *menolak baris baru yang pemiliknya berbeda dengan role aktif. Baca hasil:* `new row violates row-level security policy`*; kegagalan command adalah hasil praktik yang benar.*

2. Lihat database dan pgvector di web

Buka http://127.0.0.1:8087/ di Chrome. Pilih **PostgreSQL**, server `db`, user `labadmin`, database `lab07`, lalu isi password dari file lokal `secrets/db_password.txt`. Pilih tabel `rls_demo` lalu **select**. Sebagai pemilik tabel, Adminer menampilkan **dua baris**. Itu bukan bukti RLS gagal: owner PostgreSQL dapat melewati RLS secara default. Bukti isolasi tetap hasil `SET ROLE` di atas.



Langkah: login Adminer sebagai `labadmin` *dan pilih* `rls_demo`*. Fungsi: melihat bentuk data melalui UI web. Cara kerja: Adminer terhubung melalui jaringan internal Compose ke* `db`*; login owner punya konteks hak berbeda dari Alice/Bob. Baca hasil: dua baris terlihat; bandingkan dengan screenshot query per-role.*

Jalankan contoh vektor mainan:

PowerShell:

```
Get-Content -Raw .\vector-local-demo.sql | docker compose exec -T db psql -v ON_ERROR_STOP=1 -U
labadmin -d lab07
```

Bash/Linux/Git Bash:

```
docker compose exec -T db psql -v ON_ERROR_STOP=1 -U labadmin -d lab07 < vector-local-demo.sql
```

`CREATE EXTENSION vector` mengaktifkan tipe `vector(3)`. Operator `<->` menghitung jarak Euclidean terhadap query `[1,0,0]`. Hasil terdekat harus `Docker` (jarak 0), lalu `Container` (sekitar 0,141), dan `Database` (sekitar 1,414). Ini **bukan embedding model**; Lab 11 memakai embedding untuk RAG.

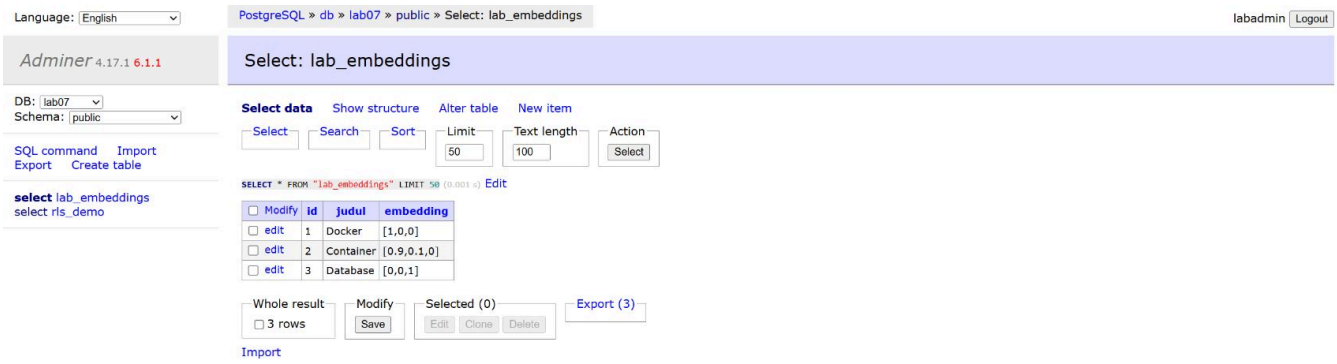
```
LAB 07 ? urutan jarak vektor

$ docker compose exec -T db psql ... -c "SELECT judul, jarak ... ORDER BY embedding <-> query"
judul    | jarak
----------+-------
 Docker    | 0.000
 Container | 0.141
 Database  | 1.414
(3 rows)

Cuplikan output uji lokal aktual; baris dipilih dan ditata agar terbaca.
```

Langkah: jalankan `vector-local-demo.sql`*. Fungsi: memperkenalkan pencarian tetangga terdekat. Cara kerja: PostgreSQL menghitung* `embedding <-> '[1,0,0]'` *untuk tiga baris lalu mengurutkannya. Baca hasil: Docker, Container, Database dengan jarak meningkat.*

Di Adminer, pilih tabel `lab_embeddings` untuk melihat tiga baris vektor mentah:



Langkah: klik `select lab_embeddings` *di Adminer. Fungsi: menghubungkan output SQL dengan data yang disimpan. Cara kerja: Adminer membaca tabel yang dibuat oleh script vektor. Baca hasil: kolom* `embedding` *memuat tiga vektor; urutan kemiripan tetap dibuktikan oleh query* `ORDER BY`*, bukan urutan tampilan tabel.*

3. Challenge A–F dan semua kunci

Kerjakan dahulu dari pertanyaan bila ingin latihan mandiri; jawaban lengkap berada di tabel ini.

Bagian	Tantangan kerja	Kunci dan bukti
A	Bagaimana membuktikan database siap sebelum SQL dijalankan?	`docker compose --profile debug up -d --wait` lalu `docker compose ps`; `db` harus Healthy. Screenshot 01.
B	Mengapa `GRANT SELECT` saja belum membatasi catatan per staf?	`GRANT` memberi hak pada tabel, sedangkan policy RLS memfilter baris. `ALTER TABLE rls_demo ENABLE ROW LEVEL SECURITY` dan policy `owner_name = current_user` diperlukan. Screenshot 02.
C	Mengapa Adminer menampilkan dua baris tetapi Alice/Bob masing-masing satu?	`labadmin` adalah owner tabel; uji dengan `SET ROLE lab_alice/bob`. Screenshot 03 dan 07.
D	Apa beda `USING` dan `WITH CHECK`?	`USING` memeriksa baris lama yang boleh dilihat/diubah; `WITH CHECK` memeriksa baris baru atau hasil UPDATE. INSERT Bob untuk Alice ditolak. Screenshot 04.
E	Mana hasil terdekat untuk vektor `[1,0,0]`, dan apakah ini model ML?	`Docker` jarak 0, lalu Container, Database. Ini angka contoh, bukan embedding model. Screenshot 05 dan 08.
F	Bagaimana identitas lokal berbeda dari Supabase?	Lokal memakai role `current_user`; Supabase Auth memberi JWT dan policy memakai `auth.uid()`. Jangan taruh service role/secret key di frontend.

Jalankan checker hasil akhir dari root repo:

PowerShell: `& .\tests\challenge.ps1`

Bash/Linux/Git Bash: `bash tests/challenge.sh`

Keduanya membaca metadata/query dan sengaja mencoba satu INSERT yang harus ditolak. Target **10 PASS, 0 FAIL**.



Langkah: jalankan checker sesuai terminal. Fungsi: mengulang uji kesiapan, role, RLS, pgvector, jarak, dan penolakan INSERT. Cara kerja: skrip memanggil `psql` *dalam container; penolakan INSERT dihitung PASS bila pesannya sesuai. Baca hasil:* `Lab 07: 10 PASS, 0 FAIL`.

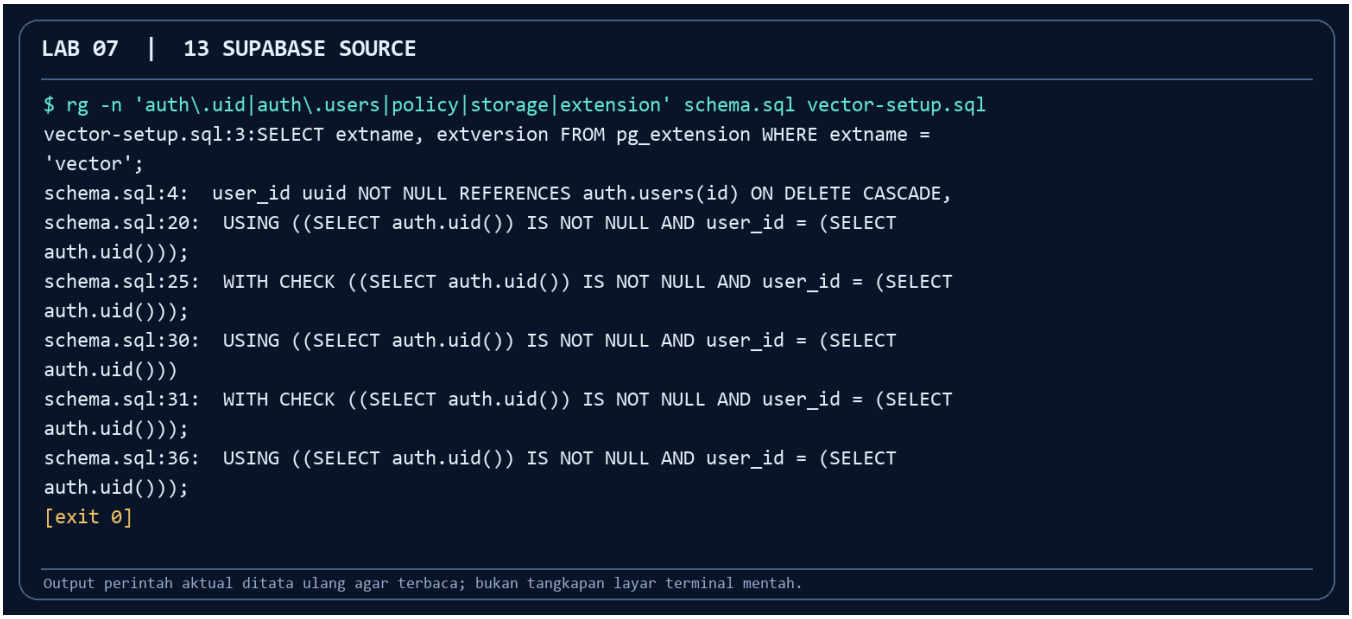
4. Jalur Supabase tambahan

Bagian ini hanya bila proyek Supabase tersedia. Jalur klien Node memakai **Node.js minimal 22.0.0** sesuai kebutuhan `@supabase/supabase-js` pada `package-lock.json` dan pengumuman dukungan Node Supabase. Jalur Docker/SQL wajib di atas tidak membutuhkan Node. Dari root repo periksa runtime terlebih dahulu:

```
node --version
npm --version
```

`node --version` harus menunjukkan `v22.0.0` atau lebih baru; `npm --version` harus menampilkan nomor versi. Jika runtime lebih tua atau instalasi menampilkan `EBADENGINE`, siapkan Node 22+ di lingkungan praktik dan buka kembali terminal sebelum mengulang.

Di **SQL Editor proyek Supabase**, jalankan `schema.sql`; file itu merujuk `auth.users` sehingga **jangan** dijalankan pada database Compose lokal. `vector-setup.sql` opsional untuk mengaktifkan extension di proyek. Buat dua akun latihan melalui Auth, salin `.env.example` ke `.env`, isi Project URL dan **publishable key**, lalu jalankan `npm ci`, `node --env-file=.env client-demo.mjs --signup` (bila akun baru), dan `npm run demo`. Ulangi dengan akun kedua. Jangan tampilkan email/password asli, service role/secret key, atau isi `.env` pada screenshot/Git.



Langkah: jalankan `rg -n 'auth\.uid|auth\.users|policy|storage|extension' schema.sql vector-setup.sql` *sebelum membuka SQL Editor. Fungsi: mengenali bagian yang membutuhkan Supabase Auth. Cara kerja: pencarian hanya membaca source; tidak mengirim SQL ke cloud. Baca hasil:* `auth.users` *menjadi rujukan pemilik, dan policy memakai* `auth.uid()` *untuk baca/tulis. Ini bukti persiapan source, bukan screenshot eksekusi cloud. Output lengkap.*

Jalur tambahan ini belum mempunyai screenshot hasil Supabase di repo karena praktiknya memerlukan proyek/akun milik peserta. Jika mengerjakannya, ambil bukti dari SQL Editor, daftar user Auth yang disamarkan, dan hasil dua akun pada mesin Anda. Jangan menganggap screenshot demo PostgreSQL lokal sebagai bukti jalur Supabase.

Di Supabase, Auth memastikan siapa pengguna. Policy `schema.sql` memakai `auth.uid()` untuk memastikan `user_id` pada baris sama dengan identitas JWT. Storage bucket latihan boleh dibuat **private**; akses objek tetap perlu policy yang dirancang. Jika email perlu verifikasi, selesaikan sebelum mencoba klien.

5. Laporan, Git, cleanup

Salin `hasil/TEMPLATE_LAPORAN.md` menjadi `hasil/lab07.md`, isi dengan penjelasanmu, dan simpan screenshot pribadi di `hasil/bukti/`. Dari repo milikmu, periksa file sebelum push:

```
git status --short
git check-ignore -v .env secrets/db_password.txt
git add README.md MODUL_MAHASISWA.md compose.yaml local-rls-demo.sql vector-local-demo.sql schema.sql
vector-setup.sql client-demo.mjs package.json package-lock.json tests hasil/lab07.md hasil/bukti
git diff --cached --name-only
git diff --cached --check
git commit -m "lab07: buktikan RLS dan jarak pgvector"
git push
```

Jika file laporan/bukti belum dibuat, buat dahulu atau sesuaikan `git add`. `.env` dan `secrets/db_password.txt` harus tetap tidak terlacak. Setelah bukti tersimpan:

LAB 07 | 11 GIT AMAN

```
$ git status --short
?? screenshots/09_setup_validasi.png
?? screenshots/09_setup_validasi.txt
?? screenshots/10_sql_setup.png
?? screenshots/10_sql_setup.txt
[exit 0]
$ git check-ignore -v .env secrets/db_password.txt
.gitignore:1:.env   .env
.gitignore:3:secrets/db_password.txt    secrets/db_password.txt
[exit 0]
$ git diff --cached --check; if ($LASTEXITCODE -eq 0) { 'Staged diff: tidak ada
whitespace error (exit 0)' }
Staged diff: tidak ada whitespace error (exit 0)
[exit 0]
```

Output perintah aktual ditata ulang agar terbaca; bukan tangkapan layar terminal mentah.

Langkah: jalankan `git status --short`, `git check-ignore -v .env secrets/db_password.txt`, *dan* `git diff --cached --check`. *Fungsi: memeriksa isi kerja sebelum commit. Cara kerja: status menampilkan perubahan, check-ignore menunjukkan aturan yang melindungi secret, dan diff check menemukan whitespace salah pada file yang sudah staged. Baca hasil: file rahasia terabaikan; cuplikan ini belum memiliki file staged, maka ulangi diff check sesudah* `git add`. *Screenshot ini belum membuktikan push; ambil bukti commit/push dari repo sendiri. Output lengkap.*

```
docker compose --profile debug down
docker compose ps
```

`down` menghentikan container; volume database tetap ada agar hasil dapat dibuka lagi. Jangan gunakan `down -v` kecuali memang ingin menghapus data latihan.

LAB 07 | 12 CLEANUP

```
$ docker compose --profile debug down
Container cloudlab07-rls-adminer-1 Stopping
 Container cloudlab07-rls-adminer-1 Stopped
 Container cloudlab07-rls-adminer-1 Removing
 Container cloudlab07-rls-adminer-1 Removed
 Container cloudlab07-rls-db-1 Stopping
 Container cloudlab07-rls-db-1 Stopped
 Container cloudlab07-rls-db-1 Removing
 Container cloudlab07-rls-db-1 Removed
 Network cloudlab07-rls_default Removing
 Network cloudlab07-rls_default Removed
[exit 0]
$ docker compose --profile debug ps
NAME      IMAGE     COMMAND   SERVICE   CREATED   STATUS    PORTS
[exit 0]
```

Output perintah aktual ditata ulang agar terbaca; bukan tangkapan layar terminal mentah.

Langkah: jalankan `docker compose --profile debug down` *lalu* `docker compose --profile debug ps`. *Fungsi: melepas container/jaringan lab seusai praktik. Cara kerja: Compose berhenti tanpa flag* `-v`, *sehingga volume tetap tersimpan. Baca hasil: container berhenti dan daftar* `ps` *kosong. Pada uji ulang,* `up -d --wait` *menghidupkan DB lagi dengan dua catatan dan tiga vektor yang tetap ada. Output lengkap.*

Jika ada kendala

Gejala	Pemeriksaan
`secrets/db_password.txt` tidak ada	Salin file contoh seperti langkah 0; jangan commit hasil salinan.
Database belum sehat	`docker compose logs db --tail 30` dan `docker compose ps`; tunggu healthcheck.
Adminer tidak terbuka	Jalankan dengan `--profile debug`; buka port **8087**, bukan 8081.
Adminer menampilkan dua baris	Itu login owner; gunakan `SET ROLE` untuk membuktikan RLS.
SQL `auth.users` tidak dikenal	`schema.sql` hanya untuk proyek Supabase. Lokal memakai `local-rls-demo.sql`.
Checker gagal di pgvector	Jalankan `vector-local-demo.sql` lalu ulangi checker.

Rujukan: Supabase RLS, vector columns, pgvector.